Enterprise Software Architecture (End-to-End)

DEVELOPED BY TALENCIAGLOBAL

ADVANCED LEVEL

A comprehensive architectural framework for designing, governing, and evolving mission-critical technology systems at scale — bridging Spring Boot, Redis, Kafka, Kubernetes, and 50+ technologies into production-grade FinTech platforms processing millions of transactions daily.

What Is Enterprise Software Architecture?

The City Planning Analogy

Think of Enterprise Software Architecture as the **"city planning" of technology.** Individual technologies — Spring Boot, Redis, Kafka — are like buildings: a restaurant, a bank, a police station. Enterprise architecture is the master plan that decides where to place each building, how to connect them with roads (APIs), ensure backup power (high availability), emergency services (monitoring), and expansion plans (scalability) — all while following building codes (compliance).

ⓘ A payment app isn't just Spring Boot. It's **50+ services orchestrated**: API Gateway, Redis, Kafka, PostgreSQL, Elasticsearch, Prometheus, Vault, and Kubernetes — all working together to process a ₹100 UPI payment in under 500ms, reliably, securely, and compliantly.

Formal Definition

Enterprise software architecture is the holistic discipline of designing, governing, and evolving technology systems that support business capabilities at scale. It encompasses:

- **Application architecture** – service design
- **Data architecture** – storage and flow
- **Infrastructure architecture** – deployment and operations
- **Security architecture** – zero-trust and compliance
- **Integration architecture** – communication patterns

Core Purpose & Business Value in FinTech

Enterprise architecture is not an academic exercise — it delivers measurable, contractual business outcomes. The following capabilities translate directly into competitive advantage and regulatory compliance for FinTech organizations.

Systematic Integration

50 technologies work as one system, not 50 isolated silos. Eliminates integration spaghetti and reduces incident resolution from 4 hours to 10 minutes.

Non-Functional Guarantee

99.99% uptime, <200ms p99 latency, 10,000 TPS — contractually delivered and continuously measured via SLOs.

Regulatory Compliance

RBI audit passes because audit trails, encryption, and data retention are designed-in — not retrofitted at 10× the cost.

Cost Optimization

₹2 crore/year cloud bill vs. ₹5 crore without architecture planning. Cost per UPI transaction: ₹0.20 (target <₹0.15).

Change Agility

Add a BNPL feature in 2 weeks (not 2 months) due to loose coupling, contract testing, and consistent service patterns.

Talent Onboarding

New engineers productive in 1 week — consistent patterns replace tribal knowledge. Reduces developer churn from 35% to 15%.

Why Does Enterprise Architecture Exist?

Real Problems It Solves in FinTech

Integration Spaghetti

Payment service calls account service calls fraud service calls notification service — a synchronous chain producing 5-second latency. Enterprise architecture adds async messaging, circuit breakers, and bulkheads to break the chain.

Technology Graveyard

2018: JBoss. 2020: WebLogic. 2022: custom RPC framework. 2024: gRPC. No standard means a maintenance nightmare. Architecture mandates technology governance and deprecation policies.

Compliance Gaps

PCI-DSS requires encryption at rest and in transit. One team forgets TLS → breach. Enterprise architecture mandates security patterns uniformly across all 80+ services.

Scaling Bottlenecks

Monolith handles 100 TPS. Split into 50 microservices but the database remains a single point — still 100 TPS. Enterprise design adds read replicas, sharding, and caching layers.

Disaster Recovery Failure

"We have DR" — but nobody tested failover for 2 years. Enterprise architecture mandates regular chaos engineering and automated failover drills, reducing RTO from 2 hours to under 15 minutes.

⚠ Without enterprise architecture: outage frequency is **monthly** (vs. annual), incident resolution takes **4 hours** (vs. 10 minutes), time-to-market for new payment methods is **6 months** (vs. 2 weeks), and compliance costs reach **₹5 crore/year** in auditor remediation fees.

Evolution & History of Enterprise Architecture

Enterprise software architecture has evolved through distinct eras, each driven by the limitations of the previous generation. FinTech unicorns like Razorpay and PhonePe were born in the microservices era — and are now navigating the platform engineering and AI-native frontier.

1990s — Monolith

1

C, COBOL, Oracle, Tuxedo. Core banking: reliable but slow to change. Deployments took weeks.

2000s — SOA / ESB

2

Java EE, WebLogic, MQ Series, SOAP. Integration achieved but ESB became the new bottleneck.

2010s — Microservices

3

Spring Boot, REST, Kafka, Docker. FinTech unicorns scaled to millions of users with independent deployments.

2015–18 — Cloud-Native

4

Kubernetes, Istio, Prometheus, Redis. 10× developer productivity. Infrastructure as code became standard.

2019–21 — Event-Driven

5

Kafka Streams, Debezium, Elasticsearch. Real-time fraud detection under 50ms. CQRS adopted widely.

2022–24 — Platform Engineering

6

Backstage, Crossplane, Internal Dev Portals. Self-service infrastructure for 200+ engineering teams.

2024–26 — AI-Native

7

Vector DBs (Pinecone), LLM gateways, WebAssembly. Personalized wealth advisory and edge fraud detection.

FinTech Maturity Model

Every FinTech organization sits at a maturity level that determines its operational resilience, deployment velocity, and compliance posture. Understanding your current level is the first step toward a structured improvement roadmap.

1	**Level 1 — Chaotic (Startup)** 1–10 services, monolith or simple microservices. "Works on my machine" deployment. Uptime: 99% (8 hours downtime/month). Time-to-recover: 2 hours.
2	**Level 2 — Managed (Growing FinTech)** 10–50 services, basic CI/CD. Monitoring via Prometheus + Grafana. Uptime: 99.9% (43 minutes/month). Time-to-recover: 30 minutes.
3	**Level 3 — Defined (Mid-size FinTech)** 50–200 services, service mesh (Istio/Linkerd). Chaos engineering, SLA-based routing. Uptime: 99.95% (21 minutes/month). Time-to-recover: 10 minutes.
4	**Level 4 — Optimized (Large Bank/FinTech)** 200+ services, platform engineering. AI-assisted root cause analysis. Uptime: 99.99% (4 minutes/month). Time-to-recover: 2 minutes.
5	**Level 5 — Anti-Fragile (Global FinTech)** Multi-region active-active, self-healing infrastructure. Predictive scaling, auto-remediation. Uptime: 99.999% (26 seconds/month). Time-to-recover: <30 seconds.

Case Study: UPI Payment Processing — End-to-End Flow

The following traces a single ₹500 UPI payment from a user's mobile device through 14 discrete technology components — completing in **80ms total**, well under the 500ms RBI SLA. This is enterprise architecture in action: 50+ technologies orchestrated as a single, coherent system.

Step	Component	Technology	Action	Latency
1	Mobile App	iOS/Android	User enters ₹500, UPI PIN	N/A
2	CDN	CloudFront	Serve app assets (cached)	10ms
3	API Gateway	Kong	Validate JWT, route to payment service	2ms
4	Rate Limiter	Redis	INCR user:123:upi → check <1000/hour	1ms
5	Auth	Keycloak	Validate OAuth2 token, extract roles	5ms
6	Circuit Breaker	Resilience4j	Check downstream health via Istio	1ms
7	Payment Service	Spring Boot	Parse request, validate amount	10ms
8	Distributed Lock	Redis	SET lock:account:456 NX PX 5000	2ms
9	Account Service	Spring Boot + Hibernate	Load account from PostgreSQL	15ms
10	Fraud Service	Python + Redis	Check ML model (pre-loaded in Redis)	8ms
11	Ledger Update	Hibernate	Update balance (optimistic lock)	20ms
12	Event Publish	Kafka	payment:completed event to stream	3ms
13	Release Lock	Redis Lua	Atomic check-and-delete	1ms
14	Response	All	Return success to mobile app	2ms

80ms

Total p99 Latency

End-to-end UPI payment completion, well under the 500ms RBI SLA

50+

Technologies Orchestrated

Working as a single coherent system across compute, cache, messaging, and observability

14

Discrete Steps

Each with defined SLA, fallback behavior, and distributed tracing via Jaeger

Enterprise Capability Matrix by Maturity Level

The following matrix maps eight critical architectural capabilities across maturity levels 2 through 5. Use this as a diagnostic tool to identify gaps and prioritize investments in your organization's architecture roadmap.

Capability	Level 2 — Managed	Level 3 — Defined	Level 4 — Optimized	Level 5 — Anti-Fragile
Service Discovery	Kubernetes DNS	Consul/etcd	Service mesh (Istio)	Multi-region mesh
Load Balancing	kube-proxy (L4)	Ingress (L7)	Envoy with traffic shifting	Global LB + GeoDNS
Resilience	Retry + timeout	Circuit breaker + bulkhead	Retry budget + client-side rate limiting	Hedged + predictive retry
Observability	Metrics (Prometheus)	Metrics + logs + traces	Continuous profiling (Pyroscope)	AI-assisted RCA + auto-remediation
Deployment	Rolling update	Blue-green	Canary + A/B testing	Progressive delivery (Flagger)
Database	Single master	Read replicas	Sharding + Vitess	Distributed SQL (CockroachDB)
Cache	Single Redis	Redis Sentinel	Redis Cluster	Redis Enterprise active-active
Security	TLS + JWT	mTLS (service mesh)	Zero-trust + OPA policies	Confidential computing (enclaves)
Compliance	Manual audits	Automated scanning	Policy-as-code	Continuous compliance attestation

Real-World Case Study: Neo-Bank at 10 Million Transactions/Day

Context & Business Problem (18 Months Ago)

Company: FinTech unicorn — 10 million customers, ₹500 crore daily transaction volume.

Problems at 1M transactions/day:

- Monolith: 45-minute deployment; 1 bug took down everything
- Database: 10TB PostgreSQL with 100ms+ queries
- Caching: Inconsistent — customers saw stale balances
- Fraud detection: Batch (5-minute lag) → ₹2 crore fraud loss
- Compliance: RBI audit failed — missing audit trails
- Uptime: 99.5% (3.6 hours downtime/month)

Current State Architecture

- **80 microservices** (Spring Boot + Python + Node.js) behind Istio service mesh
- **PostgreSQL Citus** — 64 shards by customer_id, 2TB each, 3× read replicas per shard
- **Redis Cluster** — 32 nodes, 1TB RAM, 92% cache hit ratio
- **ClickHouse** — 50 nodes for analytics; **Elasticsearch** — 20 nodes for search
- **Kafka** — 50 topics, 1 million messages/second, audit.logs retained 7 years for RBI
- **Observability** — Prometheus (10M metrics/sec), Loki (5TB logs/day), Jaeger (100k traces/sec)
- **SLOs** — 99.95% uptime, <200ms p99, <1% error rate

Transformation Journey: 12-Month Milestone Roadmap

The neo-bank's transformation from a fragile monolith to a resilient, compliant, high-throughput platform was executed in seven structured milestones over 12 months. Each milestone delivered measurable outcomes.

1

Month 1–3

Monolith → Microservices

80 services extracted via strangler pattern. Deployment time: 45 min → 5 min.

2

Month 4–6

Database Sharding

Citus (PostgreSQL) with 64 shards. Query latency: 100ms → 15ms.

3

Month 7

Redis Cluster

Cache for sessions, rate limits, fraud patterns. DB load reduced 70%.

4

Month 8–9

Event-Driven

Kafka for all state changes. Real-time fraud detection under 50ms.

5

Month 10–12

Observability + Compliance + Chaos

MTTD: 2 hours → 5 min. RBI audit: fail → pass. Uptime: 99.5% → 99.95%.

Key Lessons Learned: Database is always the bottleneck — shard early. Cache strategy must be explicit (Cache-Aside, Write-Through, Write-Behind). Async everything not on the critical path. Contract testing (Pact) reduced integration breaks by 80%. Compliance starts at design — retrofitting costs 10× more.

Enterprise Integration Patterns

Enterprise integration patterns are the vocabulary of distributed systems design. Each pattern addresses a specific failure mode or scalability constraint encountered in production FinTech environments.

1	**API Gateway** Single entry point routing 50 services with unified auth. Kong routes to payment, account, and fraud services — eliminating 50 separate auth implementations.
2	**Circuit Breaker** Stops cascading failures when downstream error rate exceeds 50%. Resilience4j wraps all NPCI API calls — preventing a payment gateway outage from taking down the entire platform.
3	**Saga (Orchestration)** Distributed transactions across 5 services with compensating transactions. Payment → Account → Fraud → Ledger — with automatic rollback if any step fails.
4	**CQRS** Separate command (write) and query (read) paths. PostgreSQL for writes; Elasticsearch for transaction search — enabling 10,000 TPS writes without degrading read performance.
5	**Change Data Capture (CDC)** Debezium reads PostgreSQL WAL → Kafka → ClickHouse. Synchronizes OLTP to analytics in real time without application-layer coupling.
6	**Strangler Fig** Replace monolith gradually by routing requests to new services incrementally: 5% → 50% → 100% traffic. Account module extracted over 6 weeks with zero downtime.

Architectural Decision Records (ADRs)

Architectural Decision Records formalize the reasoning behind critical technology choices. They prevent re-litigation of settled decisions and provide onboarding context for new engineers. The following two ADRs represent decisions with direct FinTech compliance and performance implications.

ADR 001: Cache Strategy for User Sessions

Context: 10 million users, each session 2KB. Storing in PostgreSQL adds 50ms latency per request.

Decision: Redis Cluster (32 nodes, 1TB) with Cache-Aside pattern.

- Get session: Check Redis → if miss, load from PostgreSQL → store with 2-hour TTL
- Update session: Update PostgreSQL → invalidate Redis key → next read repopulates

Alternatives rejected: Hazelcast (not persistent), Memcached (no replication), Database-only (50ms latency).

Result: 2ms vs. 50ms latency. 92% cache hit rate. PCI-DSS session timeout enforced via Redis TTL.

ADR 002: Synchronous vs. Asynchronous Fraud Check

Context: Fraud ML inference takes 50ms. Payment critical path currently 80ms total. Deep learning models take 200ms.

Decision: Synchronous for real-time fraud (simple model, 90% accuracy); Async for deep learning (complex model, 99% accuracy, 5-minute lag).

- Sync path: Payment → Fraud Check (50ms) → Account Debit → Success
- Async path: Payment → Account Debit → Kafka → Batch ML (5 min) → Freeze if detected

Rationale: RBI mandates real-time fraud detection for UPI (<500ms). Deep learning at 200ms would violate the SLA. Trade-off accepted: 10% lower accuracy on the critical path.

Non-Functional Requirements & Trade-Offs

Non-functional requirements (NFRs) are the contractual obligations of enterprise architecture. In Tier 1 FinTech, each NFR carries direct financial, regulatory, and reputational consequences. The difference between 99.9% and 99.99% uptime is ₹100 crore in trading losses.

NFR	Target (Tier 1)	Architectural Impact	Trade-Offs
Availability	99.99% (4 min/month)	Multi-AZ, multi-region, load balancers, replication	Cost 2–3× vs. 99.9% architecture
Latency (p99)	<200ms API; <50ms trading	Edge caching, read replicas, async processing	Caching introduces stale data risk
Throughput	10,000 TPS sustained	Sharding, message queues, connection pooling	Increased complexity; harder debugging
Consistency	Strong for balances; eventual for reports	Distributed transactions (Saga), CRDTs for caches	Strong consistency = 40% lower throughput
Durability	RPO=0, RTO<15min	Synchronous replication, WAL archiving	Write latency 2× vs. async replication
Security	Zero-trust, encryption everywhere	mTLS, service mesh, Vault, CORS, CSP	30% performance overhead
Compliance	RBI, PCI-DSS, GDPR, DPDP	Audit trails, encryption, data localization	Storage costs 2× (redundancy + retention)

CAP Theorem in FinTech: UPI Payment Ledger → CP (no double spend). Transaction History → AP (5s lag acceptable). Real-time Fraud → AP (false positive better than unavailable). Trading Order Book → CP (no stale prices).

Compliance & Regulatory Architecture

Regulatory compliance in Indian FinTech is not optional — it is a license-to-operate requirement. The following maps each major regulation to its specific technology implementation, ensuring compliance is designed-in rather than retrofitted.

Regulation	Requirement	Technology Implementation
RBI (KYC)	7-year audit trail	Kafka with retention.ms=220752000000 (7 years) → S3 cold storage with SHA-256 integrity hashing
RBI (Data Localization)	Customer data in India only	Kubernetes node affinity + AWS region = ap-south-1; cross-region replication blocked at network policy level
PCI-DSS 3.2.1	Encrypt PAN at rest and in transit	HashiCorp Vault for key management + TLS 1.3 + AES-256 encryption at application layer
PCI-DSS 10.3	Audit logs with user identity	OpenTelemetry + structured logging with userId, traceId, and IP fields in every log event
GDPR Article 17	Right to erasure	Tombstone records in Kafka + anonymization service triggered by deletion request API
DPDP Act (India)	Consent management	OpenPolicyAgent policies checking consent flags before any data processing operation

Audit Trail Pipeline

Application Log → Fluentd (enrich with traceId, userId, IP) → Kafka (immutable audit topic) → Loki (3 months), S3 (7 years), Elasticsearch (6 months search), Hive (analytics)

Integrity Guarantee

SHA-256 hashes chained across audit records (blockchain-like). RBAC: only designated auditors can query S3 archive. Immutability enforced at storage policy level.

Compliance Cost Impact

Designed-in compliance: ₹1 crore/year. Retrofitted compliance: ₹5 crore/year. Violation penalty under DPDP Act: up to ₹100 crore. Architecture ROI is unambiguous.

Operational Excellence: Production Readiness

Production readiness is the operational contract between engineering and the business. The following checklist represents the minimum standard for any service handling financial transactions in a regulated environment.

Health Checks & Graceful Shutdown

Kubernetes liveness and readiness probes on every service. Spring Boot `server.shutdown=graceful` ensures zero dropped requests during rolling updates. Load tested during every deployment.

Observability (RED Metrics)

Rate, Errors, Duration tracked via Micrometer + Prometheus for every service. Grafana dashboards mandatory before production promotion. SLO-based alerting via Alertmanager → PagerDuty escalation.

Secrets & Dependency Governance

HashiCorp Vault auto-rotates secrets every 90 days via Kubernetes CSI. Dependabot + Renovate enforce critical CVE patches within 7 days. Audit log verification on every rotation.

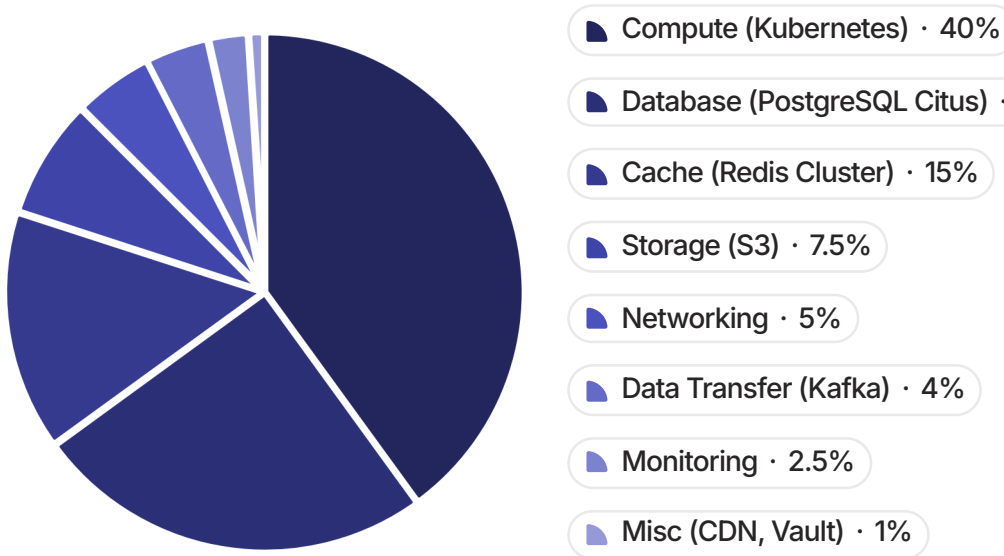
Backup & DR Drills

pg_dump + WAL archiving with monthly restore drills. Cross-region failover tested quarterly via Terraform + Chaos Mesh GameDays. RTO target: <15 minutes. Uptime improved from 99.5% to 99.95% after first GameDay cycle.

On-Call Runbook (High Latency p99 >500ms): 1. Identify service via Grafana. 2. Trace time distribution via Jaeger. 3. Check slow queries via pg_stat_statements. 4. Check Redis hit ratio via INFO. 5. Check GC pauses via GCeasy. Common fixes: increase connection pool, shard Redis hot key, add JOIN FETCH, tune G1GC → ZGC, increase Kubernetes CPU request.

Cloud Cost Architecture: 10M Transactions/Day

Cost governance is a first-class architectural concern. Without deliberate cost architecture, a 10-team FinTech can waste ₹50 lakh/month on redundant Redis clusters alone. The following breakdown represents a ₹2 crore/month optimized cloud bill for a platform processing 10 million transactions daily.



Cost Optimization Levers

- **Compute (₹80L/month):** Spot instances for batch workloads; right-size CPU/memory requests via VPA
- **Database (₹50L/month):** 3-year reserved instances + auto-scaling down at night
- **Cache (₹30L/month):** Tiered memory (RAM + SSD); aggressive TTL policies; maxmemory-policy
- **Storage (₹15L/month):** S3 Intelligent-Tiering + lifecycle policies for audit archives
- **Kafka (₹8L/month):** Snappy compression reduces storage 40%

Cost per transaction: ₹0.20 (2 paise). Target: <₹0.15. Each 1% improvement in Redis cache hit ratio saves ₹3 lakh/month. Spot instances for fraud ML training save 70%.

Maturity Self-Assessment & 12-Month Roadmap

Self-Assessment Tool (Score 1–5 per Category)

Category	Level Progression
Deployment	Manual → CI/CD → Blue-green → Canary → Progressive
Observability	Logs → Metrics → Traces → Profiling → AI RCA
Resilience	Retry → Timeout → Circuit breaker → Bulkhead → Hedged
Scalability	Vertical → Read replicas → Sharding → Auto → Predictive
Security	TLS → JWT → mTLS → Zero-trust → Confidential compute
Compliance	Manual → Automated → Policy-as-code → Continuous → Immutable
Cost	Unmanaged → Tagging → Budget alerts → Auto-optimize → FinOps

Score interpretation: 7–17 (Startup), 18–35 (Growing), 36–70 (Mature), 71–105 (World-class)

Sample 12-Month Roadmap: Startup → Enterprise

Month 1–3: Foundation

- Monolith → 10 microservices
- CI/CD (GitHub Actions → EKS)
- PostgreSQL with read replica
- Prometheus + Grafana baseline

Month 4–6: Resilience

- Circuit breakers (Resilience4j)
- Redis Cluster for caching
- SLOs targeting 99.9% uptime

Month 7–9: Scale

- Database sharding (Citus)
- Kafka for async events
- Service mesh (Istio) + HPA

Month 10–12: Excellence

- Multi-region active-passive
- Weekly chaos engineering GameDays
- Compliance automation (OPA)
- 99.99% uptime + FinOps (–30% cost)

Comparison: Enterprise Architecture vs. Alternatives

The choice of architectural approach has compounding consequences over a 12–24 month horizon. The following comparison evaluates enterprise architecture against two common failure modes: "just code it" (no architecture) and over-engineering (Big Design Up Front).

Aspect	Enterprise Architecture	"Just Code It" (No Architecture)	Over-Architected (BDUF)
Time-to-market	3 months initial; 2 weeks per feature	1 month initial; 4 weeks per feature	6 months initial; never shipped
Technical debt	Managed (scheduled refactoring)	Crippling (50% of time firefighting)	Low but progress is paralyzed
Team productivity	High (consistent patterns)	Low (constant context switching)	Low (analysis paralysis)
Onboarding time	1 week	3 weeks	2 weeks
Scalability ceiling	100,000+ TPS	500 TPS	Unknown (never shipped)
Compliance readiness	Designed-in	Retrofit (10× cost)	Perfect but late
Cloud cost	₹2 crore/month (optimized)	₹5 crore/month (wasteful)	Unknown (over-provisioned)

> *"Architecture is not about doing more work. It's about doing the right work once, so 100 engineers don't do it wrong 100 times."*

Integration Complexity Spectrum & Decision Framework

Selecting the appropriate integration pattern is one of the most consequential architectural decisions. Over-engineering a simple REST API with Kafka adds latency and operational overhead. Under-engineering a high-throughput payment pipeline with synchronous REST causes cascading failures. The following framework provides a structured decision path.

This progression reflects the real-world evolution of FinTech platforms — from a single Spring Boot service to a fully decentralized data mesh. Each level introduces new operational capabilities while adding corresponding complexity that must be justified by scale and business requirements.

<10 Services, <100 TPS

Simple REST. No gateway overhead. Direct service-to-service calls with basic retry logic.

10–50 Services, <1,000 TPS

API Gateway + Resilience patterns (circuit breaker, bulkhead, retry with exponential backoff).

50+ Services, 1,000+ TPS

Async messaging (Kafka) + CQRS + Event Sourcing. Decouple producers from consumers for independent scaling.

Low Latency (<10ms)

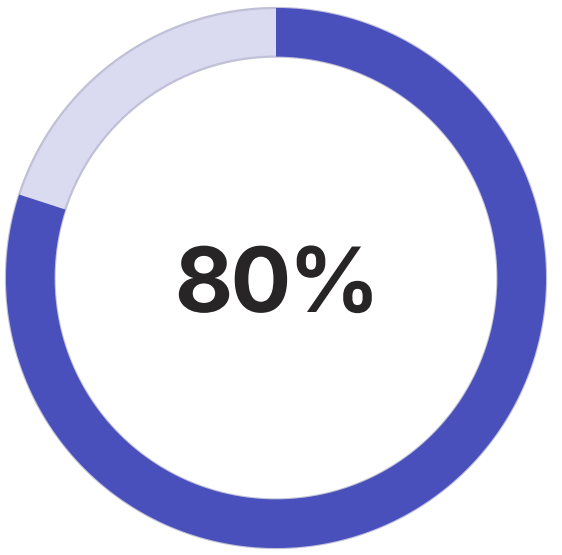
Synchronous only — no messaging overhead. Reactive (WebFlux + RSocket) for non-blocking I/O at microsecond scale.

Summary: The Enterprise Architecture Imperative

Enterprise software architecture is the disciplined practice of making thousands of small, deliberate decisions about technology, patterns, trade-offs, and processes — so that a ₹100 UPI payment works reliably for 10 million customers, 24×7, while passing RBI audits, costing ₹0.20 per transaction, and enabling 100 engineers to deploy safely 50 times per day.

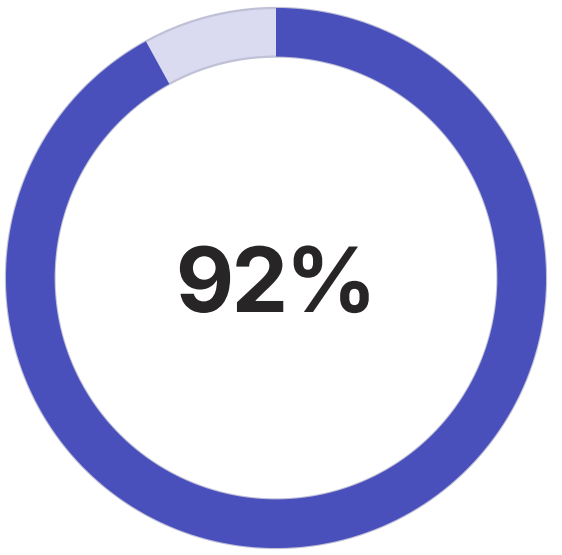
Target Uptime

4 minutes downtime per month — the difference between ₹100 crore in trading losses and business continuity



Integration Break Reduction

Achieved via Pact contract testing between services — from weekly weekend incidents to monthly planned maintenance



Cache Hit Ratio

Redis Cluster performance in production — each 1% improvement saves ₹3 lakh/month in database compute costs

It Is Not About Every Technology

Enterprise architecture is about integrating the **right technologies** at the **right maturity level** for your stage of growth. A Level 1 startup adopting CockroachDB and Confidential Computing is over-engineering. A Level 4 bank running a monolith is a compliance liability.

Compliance Is Architecture

RBI, PCI-DSS, GDPR, and DPDP requirements are not legal constraints bolted onto engineering — they are **first-class architectural requirements** that must be designed into audit trails, encryption, data localization, and consent management from day one.

Cost Is Architecture

Without deliberate cost governance, 10 teams independently provisioning Redis clusters waste ₹50 lakh/month. FinOps is not a finance function — it is an architectural discipline requiring TTLs, spot instances, compression, and tiered storage as first-class design decisions.

DEVELOPED BY TALENCIAGLOBAL | ENTERPRISE ARCHITECTURE SERIES | ADVANCED LEVEL